

Documentação de Implementação: Etapa 3

Sistema Distribuído de Entregas com Drones

Hanny Emilyly Rodrigues Sousa

August 18, 2025

1 Visão Geral da Implementação

Nesta etapa, o projeto do Sistema Distribuído de Entregas com Drones evoluiu significativamente, saindo de uma prova de conceito básica para uma simulação mais robusta e interativa. Com base na comunicação por sockets da etapa anterior, foram introduzidos três aprimoramentos principais: uma arquitetura de mensagens formal, um mecanismo de descoberta de serviços e uma simulação de mapa com visualização gráfica.

Os principais recursos implementados foram:

1. **Arquitetura Orientada a Mensagens:** A comunicação foi padronizada com mensagens JSON estruturadas, diferenciando claramente **Comandos** (solicitações de ação) e **Eventos** (notificações de estado), o que tornou o sistema mais organizado e extensível.
2. **Mecanismo de Nomeação de Processos:** Foi criado um **Serviço de Nomes** (`servidor_nomes.py`) centralizado. Agora, os componentes não precisam mais conhecer previamente o endereço do servidor principal; eles o descobrem dinamicamente, o que torna o sistema mais flexível e desacoplado.
3. **Simulação de Mapa e Movimento:** O conceito abstrato de "entrega" foi transformado em uma simulação concreta. O servidor agora gerencia um mapa com coordenadas, e os drones simulam um movimento passo a passo nesse mapa, reportando sua posição continuamente.
4. **Visualizador Gráfico:** Para complementar a simulação de mapa, foi desenvolvido um novo componente, o **Visualizador** (`visualizador.py`), que se conecta ao servidor e exibe uma representação gráfica em tempo real do mapa, dos pontos de coleta/entrega e da posição dos drones.

2 Projeto de Arquitetura de Mensagem/Fluxo

2.1 Tipos de Mensagem: Comandos e Eventos

O sistema utiliza dois tipos de mensagens para organizar a comunicação:

- **Comandos:** Representam uma solicitação para que um componente execute uma ação. São sempre iniciados por um cliente (drone, cliente de pedidos, visualizador) e direcionados ao servidor.

– `comando_registrar_servico`

- comando_consultar_servico
- comando_registrar_drone
- comando_solicitar_entrega
- comando_entrega_finalizada
- comando_atualizar_posicao
- comando_solicitar_estado_mapa

- **Eventos:** Representam uma notificação sobre algo que aconteceu ou uma resposta a um comando. Geralmente são emitidos pelo servidor.

- evento_pedido_confirmado
- evento_pedido_rejeitado
- evento_pedido_atribuido
- evento_estado_mapa

2.2 Estrutura e Formato das Mensagens-Chave

Todas as mensagens seguem um formato JSON padrão com os campos `tipo` e `payload`.

```

1 {
2   "tipo": "comando_solicitar_entrega",
3   "payload": {
4     "origem": "P1",
5     "destino": "D3"
6   }
7 }
```

Listing 1: Comando para Solicitar Entrega (Cliente → Servidor)

```

1 {
2   "tipo": "evento_pedido_atribuido",
3   "payload": {
4     "id": "f3a1b8c2",
5     "origem": {
6       "nome": "P1",
7       "pos": [2, 2]
8     },
9     "destino": {
10      "nome": "D3",
11      "pos": [3, 7]
12    },
13    "drone_posicao_inicial": [0, 0]
14  }
15 }
```

Listing 2: Evento de Atribuição de Pedido (Servidor → Drone)

2.3 Fluxos de Comunicação Mais Importantes

O fluxo de vida de um pedido de entrega demonstra a interação entre todos os componentes:

1. **Inicialização:** O `servidor_nomes.py` é iniciado, seguido pelo `servidor.py` que se registra. O `visualizador.py` e os drones consultam o serviço de nomes para encontrar e se conectar ao servidor principal.

2. **Disponibilidade:** Cada drone envia um `comando_registrar_drone`. O servidor os adiciona ao seu pool de drones disponíveis.
3. **Criação do Pedido:** O `cliente.py` envia um `comando_solicitar_entrega`.
4. **Validação e Enfileiramento:** O servidor valida o pedido, o coloca na fila e responde com um `evento_pedido_confirmado`.
5. **Atribuição:** O servidor seleciona um drone disponível aleatoriamente e lhe envia um `evento_pedido_atribuido`.
6. **Execução e Rastreamento:** O drone se move, enviando um `comando_atualizar_posicao` a cada passo.
7. **Visualização (Fluxo Paralelo):** O visualizador solicita e recebe o estado do mapa (`evento_estado_mapa`) continuamente para redesenhar a tela.
8. **Finalização:** O drone envia um `comando_entrega_finalizada` e o servidor o marca como disponível novamente.

2.4 Fluxo Contínuo de Dados

O projeto aborda a noção de fluxo contínuo de dados através da interação entre os drones em movimento e o servidor. O envio constante de mensagens `comando_atualizar_posicao` por parte do drone cria um *stream* de eventos de localização. Esse fluxo é consumido pelo servidor para manter o estado do mapa sempre atualizado, e é essencial para a funcionalidade do visualizador.

3 Mecanismo de Nomenclatura do Processo

3.1 Descrição do Mecanismo

Foi implementado um mecanismo de **Registro Centralizado**, o **Serviço de Nomes** (`servidor_nomes.py`). Esta abordagem desacopla os componentes, eliminando a necessidade de endereços de IP e portas fixos no código dos clientes.

3.2 Registro e Descoberta de Componentes

- **Registro:** Um serviço (o `servidor.py`) envia uma mensagem `comando_registrar_servico` ao Serviço de Nomes ao ser inicializado, contendo um nome lógico e sua porta.
- **Descoberta:** Um componente (drone, cliente) envia uma mensagem `comando_consultar_servico` ao Serviço de Nomes para obter o endereço IP e a porta do serviço desejado.

3.3 Informações Armazenadas e Acesso

O Serviço de Nomes armazena os dados em um dicionário Python em memória. A chave é o nome do serviço e o valor é uma tupla (IP, porta). O acesso a este dicionário é sincronizado usando um `threading.Lock` para garantir a segurança em concorrência.

4 Instruções de Construção e Execução

4.1 Pré-requisitos

É necessário ter o Python instalado e a biblioteca `pygame`.

```
1 pip install pygame
```

4.2 Ordem de Execução

Abra múltiplos terminais e execute os comandos na seguinte ordem:

1. Serviço de Nomes:

```
1 python servidor_nomes.py  
2
```

2. Servidor de Entregas:

```
1 python servidor.py  
2
```

3. Visualizador Gráfico:

```
1 python visualizador.py  
2
```

4. Drones (um terminal por drone):

```
1 python drone.py DroneA  
2 python drone.py DroneB  
3
```

5. Cliente de Pedidos:

```
1 python cliente.py  
2
```